

词法分析程序的设计与实现--手工实现--设计报告

2023211304班 周柄名-2023210992

实验题目、要求

1. **实验目的：**选择一种源语言（本实验选择**C语言**），使用C++完成词法分析程序。
2. **输入：**源程序字符流。
3. **输出：**以记号的形式输出每个单词符号。
4. **要求：**
 - a. 可以识别并跳过源程序中的注释。
 - b. 可以统计源程序中的语句行数、各类单词的个数、以及字符总数，并输出统计结果。
 - c. 检查源程序中存在的词法错误，并报告错误所在的位置。
 - d. 对源程序中出现的错误进行适当的恢复，使词法分析可以继续进行，对源程序进行一次扫描，即可检查并报告源程序中存在的所有词法错误。

程序设计说明

一、词法规则

1. 基本保留字：

“auto”，“break”，“case”，“char”，“const”，“continue”，

“default”，“do”，“double”，“else”，“enum”，“extern”，

“float”，“for”，“goto”，“if”，“inline”，“int”，

“long”，“register”，“restrict”，“return”，“short”，“signed”，

“sizeof”，“static”，“struct”，“switch”，“typedef”，“union”，

“unsigned”，“void”，“volatile”，“while”，“_Alignas”，“_Alignof”，

“_Atomic”，“_Bool”，“_Complex”，“_Generic”，“_Imaginary”，“_Noreturn”，

“_Static_assert”，“_Thread_local”

2. **标识符**：所有的变量名和函数名，包括字符串(不包含中文，遇到中文直接输出error)。

3. **常数**：所有无符号实数，包括科学计数法的实数。

4. **运算符**：

```
'->' '.,' '!' '~' '++' '-' '-' '*' '&' '/' '%' '+' '-' '<<' '>>' '<'
'<=' '>' '>=' '==' '!=' '^' '|' '&&' '||' '?:' '+=' '-=' '*=' '/='
```

5. **分隔符**：

```
() {} " ' , ; [ ]
```

```
// C 语言关键字列表
const char* keywords[] = {
    "auto", "break", "case", "char", "const", "continue",
    "default", "do", "double", "else", "enum", "extern",
    "float", "for", "goto", "if", "inline", "int",
    "long", "register", "restrict", "return", "short", "signed",
    "sizeof", "static", "struct", "switch", "typedef", "union",
    "unsigned", "void", "volatile", "while", "_Alignas", "_Alignof",
    "_Atomic", "_Bool", "_Complex", "_Generic", "_Imaginary", "_Noreturn",
    "_Static_assert", "_Thread_local"
};
//运算符一类
const char operator_1[4] = { '.,', '~', '?', ':' };
//运算符二类(后面可能会接=)
const char operator_2[6] = { '!', '*', '/', '%', '^', '=' };
```

二、错误处理

1. 错误检测

- 在**预处理阶段**，程序通过状态机的方式跟踪注释的开始和结束，当遇到 `/*` 时进入注释状态，持续读取字符直到遇到 `*/` 或文件结束，如果文件结束仍未找到 `*/` 则报告未闭合注释错误。
- 在**词法分析阶段**，程序通过字符分类和状态跟踪来检测错误：当遇到双引号时进入字符串状态，持续读取直到遇到结束引号或换行符，如果换行符先出现则报告未闭合字符串错误；当识别数字时，程序检查数字后是否跟着非法字符（如字母），如果发现则报告无效数字错误；对于无法识别的字符，程序会报告非法字符错误。这种检测方式确保了程序能够发现源程序中的所有词法错误，同时不会遗漏任何错误类型。

2. 错误定位

程序通过维护精确的行号和列号信息来实现错误定位。在文件读取过程中，程序使用 `row` 和 `col` 变量分别跟踪当前的行号和列号，每当遇到换行符时 `row` 自增并将 `col` 重置为1，遇到其他字符时 `col` 自增。当检测到错误时，程序将当前的行号和列号信息保存到 `ErrorInfo` 结构体中，包括错误类型、行

号、列号等字段。对于跨行的错误（如未闭合的字符串），程序会记录错误开始时的行号和列号，确保能够准确定位错误的起始位置。错误报告时，程序使用 printf 函数输出格式化的错误信息，格式为"词法错误 [行X, 列Y]"，其中X和Y分别是错误所在的行号和列号。这种定位方式不仅准确，而且为用户提供了清晰的错误位置信息，便于快速定位和修复错误。

3. 跳过错误继续进行词法分析

当检测到错误时，程序会根据错误类型采用不同的跳过策略：对于未闭合的字符串错误，程序会跳过到行末或下一个引号，然后继续分析；对于未闭合的注释错误，程序会跳过到行末，然后继续分析；对于其他类型的错误，程序会跳过当前字符，然后继续读取下一个字符。在跳过错误的过程中，程序会更新行号和列号信息，确保后续错误定位的准确性。程序还维护了分析状态，如是否在字符串中、是否在语句中等，确保状态的一致性。通过这种恢复机制，程序能够从错误中恢复并继续分析，最终完成对整个源程序的词法分析，同时提供完整的错误统计信息。这种设计确保了程序的健壮性，即使源程序存在多个错误，程序也能够完成分析并报告所有错误。

三、使用说明

- 为了方便运行.exe文件的演示，项目最后版本将默认地址改为如右地址，分别是：
 - 源程序
 - 预处理后的源程序
 - 词法分析结果
 - 所有输出综合

```
char yuan[100] = "raw.txt";
char yuchli[100] = "pretreatment.txt";
char cifa[100] = "lexical_analysis.txt";
char output[100] = "output.txt";
```

测试报告

测试程序一：

```
#include <stdio.h>

int main() {
    int a = 10;
    float b = 3.14f;
    char c = 'A';
    char str[] = "Hello World!";

    printf("int: %d\n", a);
    printf("float: %.2f\n", b);
    printf("char: %c\n", c);
    printf("string: %s\n", str);

    return 0;
}
```

- 词法分析结果：

```

开始词法分析：
(#, 分隔符)
(include, 标识符)
(<, 运算符)
(stdio, 标识符)
(., 运算符)
(h, 标识符)
(>, 运算符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
()), 分隔符)
({, 分隔符)
(int, 基本保留字)
(a, 标识符)
(=, 运算符)
(10, 常数)
(;, 分隔符)
(float, 基本保留字)
(b, 标识符)
(=, 运算符)
(3.14, 常数)
(f, 标识符)
(;, 分隔符)
(char, 基本保留字)
(c, 标识符)
(=, 运算符)
(', 分隔符)
(A, 标识符)
(', 分隔符)
(;, 分隔符)
(char, 基本保留字)
(str, 标识符)
([, 分隔符)
()], 分隔符)
(=, 运算符)
(", 分隔符)
>Hello, 标识符)
>World, 标识符)
>(!, 运算符)
>(", 分隔符)
>(;, 分隔符)
>(printf, 标识符)
>((, 分隔符)
>(", 分隔符)
>(int, 基本保留字)

```

```

(:, 运算符)
(%, 运算符)
(d, 标识符)
(\, 分隔符)
(n, 标识符)
(", 分隔符)
(., 分隔符)
(a, 标识符)
(), 分隔符)
(;, 分隔符)
(printf, 标识符)
((, 分隔符)
(", 分隔符)
(float, 基本保留字)
(:, 运算符)
(%, 运算符)
(., 运算符)
(2, 常数)
(f, 标识符)
(\, 分隔符)
(n, 标识符)
(", 分隔符)
(., 分隔符)
(b, 标识符)
(), 分隔符)
(;, 分隔符)
(printf, 标识符)
((, 分隔符)
(", 分隔符)
(char, 基本保留字)
(:, 运算符)
(%, 运算符)
(c, 标识符)
(\, 分隔符)
(n, 标识符)
(", 分隔符)
(., 分隔符)
(c, 标识符)
(), 分隔符)
(;, 分隔符)
(printf, 标识符)
((, 分隔符)
(", 分隔符)
(string, 标识符)
(:, 运算符)
(%, 运算符)

```

```

(c, 标识符)
(\, 分隔符)
(n, 标识符)
(", 分隔符)
(., 分隔符)
(c, 标识符)
(), 分隔符)
(;, 分隔符)
(printf, 标识符)
((, 分隔符)
(", 分隔符)
(string, 标识符)
(:, 运算符)
(%, 运算符)
(s, 标识符)
(\, 分隔符)
(n, 标识符)
(", 分隔符)
(., 分隔符)
(str, 标识符)
(), 分隔符)
(;, 分隔符)
(return, 基本保留字)
(0, 常数)
(;, 分隔符)
{, 分隔符)
词法分析完成

```

测试程序二：带有注释的程序

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 全局变量定义
int global_counter = 0; // 全局计数器
const float PI = 3.14159f; // 圆周率常量

// 函数声明
int calculate_sum(int a, int b);
void print_array(int arr[], int size);
int factorial(int n);

// 主函数
int main() {
    // 局部变量声明
    int num1 = 10, num2 = 20; // 两个整数变量
    float result = 0.0f; // 浮点数结果
    char message[] = "Hello World!"; // 字符串数组

    // 单行注释: 调用函数计算两数之和
    int sum = calculate_sum(num1, num2);

    /*
    * 多行注释:
    * 这是一个复杂的计算过程
    * 包含多个步骤
    */
    result = (float)sum / 2.0f; // 类型转换
    global_counter++; // 递增全局计数器

    // 条件判断
    if (result > 15.0f) {
        printf("结果大于15: %.2f\n", result);
    } else {
        printf("结果小于等于15: %.2f\n", result);
    }

    // 循环结构
    for (int i = 0; i < 5; i++) {
        printf("循环次数: %d\n", i);
    }
}

```

```

// 循环结构
for (int i = 0; i < 5; i++) {
    printf("循环次数: %d\n", i);
}

// 数组操作
int numbers[] = {1, 2, 3, 4, 5};
print_array(numbers, 5);

// 递归函数调用
int fact = factorial(5);
printf("5的阶乘: %d\n", fact);

return 0; // 程序正常结束
}

// 计算两数之和的函数
int calculate_sum(int a, int b) {
    return a + b; // 返回两数之和
}

// 打印数组元素的函数
void print_array(int arr[], int size) {
    printf("数组元素: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// 计算阶乘的递归函数
int factorial(int n) {
    if (n <= 1) {
        return 1; // 递归终止条件
    }
    return n * factorial(n - 1); // 递归调用
}

```

• 分析结果:

开始词法分析:

(#, 分隔符)
(include, 标识符)
(<, 运算符)
(stdio, 标识符)
(., 运算符)
(h, 标识符)
(>, 运算符)
(#, 分隔符)
(include, 标识符)
(<, 运算符)
(stdlib, 标识符)
(., 运算符)
(h, 标识符)
(>, 运算符)
(#, 分隔符)
(include, 标识符)
(<, 运算符)
(string, 标识符)

(;, 分隔符)
(int, 基本保留字)
(factorial, 标识符)
((, 分隔符)
(int, 基本保留字)
(n, 标识符)
()), 分隔符)
(;, 分隔符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
()), 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)
(10, 常数)
(., 分隔符)
(int, 基本保留字)

(10, 常数)
(., 分隔符)
(num2, 标识符)
(=, 运算符)
(20, 常数)
(;, 分隔符)
(float, 基本保留字)
(result, 标识符)
(=, 运算符)
(0.0, 常数)
(size, 标识符)
()), 分隔符)
(;, 分隔符)
(int, 基本保留字)
(factorial, 标识符)
((, 分隔符)
(int, 基本保留字)
(n, 标识符)
()), 分隔符)

(., 运算符)
(h, 标识符)
(int, 基本保留字)
(global_counter, 标识符)
(=, 运算符)
(0, 常数)
(;, 分隔符)
(const, 基本保留字)
(float, 基本保留字)
(PI, 标识符)
(=, 运算符)
(3.14159, 常数)
(f, 标识符)
(;, 分隔符)
(int, 基本保留字)
(calculate_sum, 标识符)
((, 分隔符)
(int, 基本保留字)
(a, 标识符)
(,, 分隔符)
(int, 基本保留字)
(b, 标识符)
(, 分隔符)
(;, 分隔符)
(void, 基本保留字)
(print_array, 标识符)
((, 分隔符)
(int, 基本保留字)
(arr, 标识符)
([, 分隔符)
(], 分隔符)
(,, 分隔符)
(int, 基本保留字)
(size, 标识符)
(, 分隔符)
(;, 分隔符)
(int, 基本保留字)
(factorial, 标识符)
((, 分隔符)

(size, 标识符)
(, 分隔符)
(;, 分隔符)
(int, 基本保留字)
(factorial, 标识符)
((, 分隔符)
(int, 基本保留字)
(n, 标识符)
(, 分隔符)
(;, 分隔符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
(, 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)
(10, 常数)
(,, 分隔符)
(,, 分隔符)
(int, 基本保留字)
(size, 标识符)
(, 分隔符)
(;, 分隔符)
(int, 基本保留字)
(factorial, 标识符)
((, 分隔符)
(int, 基本保留字)
(n, 标识符)
(, 分隔符)
(;, 分隔符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
(, 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)

(;, 分隔符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
(, 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)
(10, 常数)
(,, 分隔符)
(num2, 标识符)
(=, 运算符)
(20, 常数)
(;, 分隔符)
(float, 基本保留字)
(result, 标识符)
(=, 运算符)
(0.0, 常数)
(factorial, 标识符)
((, 分隔符)
(int, 基本保留字)
(n, 标识符)
(, 分隔符)
(;, 分隔符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
(, 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)
(10, 常数)
(,, 分隔符)
(num2, 标识符)
(=, 运算符)
(20, 常数)
(;, 分隔符)

(float, 基本保留字)
(result, 标识符)
(=, 运算符)
(0.0, 常数)
(f, 标识符)
(;, 分隔符)
(char, 基本保留字)
(printf, 标识符)

(num2, 标识符)
(=, 运算符)
(20, 常数)
(;, 分隔符)
(float, 基本保留字)
(result, 标识符)
(=, 运算符)
(0.0, 常数)


```

(message, 标识符)
([, 分隔符)
(], 分隔符)
(,), 分隔符)
(;;, 分隔符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
(,), 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)
(10, 常数)
(,, 分隔符)
(num2, 标识符)
(=, 运算符)
(20, 常数)
(;;, 分隔符)
(float, 基本保留字)
(result, 标识符)
(=, 运算符)
(0.0, 常数)
(f, 标识符)
(;;, 分隔符)
(char, 基本保留字)
(message, 标识符)
([, 分隔符)
(], 分隔符)
((, 分隔符)
(,), 分隔符)
({, 分隔符)
(int, 基本保留字)
(num1, 标识符)
(=, 运算符)
(10, 常数)
(,, 分隔符)
(num2, 标识符)
(=, 运算符)

```

```

(f, 标识符)
(;;, 分隔符)
(char, 基本保留字)
(message, 标识符)
([, 分隔符)
(], 分隔符)
(=, 运算符)
词法分析完成

```

===== 统计结果 =====

```

总行数: 18
(f, 标识符)
(;;, 分隔符)
(char, 基本保留字)
(message, 标识符)
([, 分隔符)
(], 分隔符)
(=, 运算符)
词法分析完成

```

===== 统计结果 =====

```

总行数: 18
(=, 运算符)
词法分析完成

```

===== 统计结果 =====

```

总行数: 18
语句行数: 3
关键字个数: 15

```

===== 统计结果 =====

```

总行数: 18
语句行数: 3
关键字个数: 15
标识符个数: 26
常数个数: 5
语句行数: 3
关键字个数: 15
标识符个数: 26
常数个数: 5
运算符个数: 15
分隔符个数: 26

```



```
(20, 常数)
(;;, 分隔符)
(float, 基本保留字)
(result, 标识符)
(=, 运算符)
(0.0, 常数)
(f, 标识符)
(;;, 分隔符)
(char, 基本保留字)
(message, 标识符)
([, 分隔符)
```

```
分隔符个数: 26
标识符个数: 26
常数个数: 5
运算符个数: 15
分隔符个数: 26
运算符个数: 15
分隔符个数: 26
字符总数: 166
分隔符个数: 26
字符总数: 166
字符总数: 166
```

```
=====
```

测试程序三：带有错误的程序

raw_test3.txt

```
1  #include <stdio.h>
2
3  int main() {
4      int x = 10;
5      int y = 20;
6
7      // 语法错误: 缺少分号
8      int z = x + y
9
10     // 语法错误: 未声明的变量
11     printf("结果: %d\n", result);
12
13     // 语法错误: 括号不匹配
14     if (x > y {
15         printf("x大于y\n");
16     }
17
18     // 语法错误: 错误的字符
19     char ch = 'a';
20     printf("字符: %c\n", ch);
21
22     return 0;
23 }
24
```

词法分析结果:

```

===== C语言词法分析器（带错误检测和恢复） =====
正在分析文件：raw_test3.txt
=====

#include <stdio.h>

int main() {
    int x = 10;
    int y = 20;

    int z = x + y

    printf("结果: %d\n", result);

    if (x > y {
        printf("x大于y\n");
    }

    char ch = 'a';
    printf("字符: %c\n", ch);

    return 0;
}

预处理完成

```

```

开始词法分析:
(#, 分隔符)
(include, 标识符)
(<, 运算符)
(stdio, 标识符)
(., 运算符)
(h, 标识符)
(>, 运算符)
(int, 基本保留字)
(main, 标识符)
((, 分隔符)
(, 分隔符)
({, 分隔符)
(int, 基本保留字)
(x, 标识符)
(=, 运算符)
(10, 常数)
(;, 分隔符)
(int, 基本保留字)
(y, 标识符)
(=, 运算符)
(20, 常数)
(;, 分隔符)
(int, 基本保留字)
(z, 标识符)
(=, 运算符)
(x, 标识符)
(+, 运算符)
(y, 标识符)
(printf, 标识符)
((, 分隔符)
词法错误 [行25, 列39]
词法分析完成

===== 统计结果 =====
总行数: 10
语句行数: 2
关键字个数: 4
标识符个数: 10
常数个数: 2
运算符个数: 7
分隔符个数: 7
字符总数: 73
词法错误总数: 1
=====

```